

Chapter 1

What is Computational Astrophysics?

Informatica gaat net zo min over computers als astronomie over telescopen.
Informatics is as little about computers as astronomy is about telescopes.

Edsger W. Dijkstra

Computational astrophysics has come a long way since the early days when astronomers began to use support devices for their calculations. In this chapter we review some key developments in this field, embrace the relatively new concept of multiscale and multiphysics simulations, and introduce the Python programming environment which we will use for the rest of this book. We do this by presenting a series of examples which may look arcane now, but will (we hope) become clearer as we progress through the text.

1.1 Computational astrophysics

Computational astrophysics bridges observation and theory. The improved quality of observations and the need for deeper understanding of the underlying physics has driven demand for higher resolution and increased physical realism in numerical simulations. With exponentially growing hardware performance, we expect that the importance of large-scale simulations will only increase, but the software must also improve to keep pace with observations and theory.

1.1.1 Origin of this book

The idea for this book originated from writing the first version of the Astrophysical MULTipurpose Software Environment (AMUSE) in 2009, which at the time we called the MULTiphysics Software Environment (MUSE; [Portegies Zwart *et al.*, 2009](#)). Since then, we have taught numerous courses in computational astrophysics based on our software, and supervised many students in research projects based on AMUSE. The notes from these lectures and the example scripts written by our students and ourselves

formed the first version of what we called the AMUSE Primer. Later the idea gelled into a book, of which you are reading the second version right now.

Writing this book was fun. Writing the software was even more fun. Writing the example scripts was the hardest thing to do. The simplest looking codes often caused us most work. Our objective was to make these codes as short and crisp as possible. Making scripts educational without losing their main purpose was our primary challenge.

1.1.2 Hands on *is* hands on

In our experience, the best way to learn is by doing. And the best way to start doing is to take an example that resembles your objective to some degree, and build on it. We therefore include a large number of examples in a separate repository within the AMUSE code distribution, with many code snippets incorporated into this book.

In practice we prefer to work as follows:

- start with a scientific question,
- identify the minimal physics needed to address the question,
- find an example script that resembles the target problem as closely as possible (or ask a large-language model for help),
- rewrite that script to make it solve our problem.

Making figures and animations can be time consuming but also very rewarding. Making a publishable figure can take as much as a day. In our examples we adopt `matplotlib`¹ (Hunter, 2007) as our graphics library. There are many alternatives to `matplotlib` for making animations or figures, but its principal advantage is its transparent interface with Python, making it easy to incorporate into AMUSE.

A simple script can be written in a few hours; a more complicated script may take days to get right. But then the long painstaking process from test script to production code starts. This generally entails a number of stages, as illustrated in Figure 1.1. The main steps include adjusting the script to one’s needs, then refactoring the script, followed by profiling and optimization. We include the refactor step to stress the importance of preserving readability of source code, which so often leads to reproducible results (see Beck & Andres, 2004, for more about refactoring).

1.1.3 What about the math?

As Albert Einstein wrote on January 7, 1943 to nine-year-old Barbara Lee Wilson (Calaprice, 2002):

Do not worry about your difficulties in Mathematics. I can assure you mine are still greater.

¹Matplotlib is well documented and publicly available as open source from http://matplotlib.org/api/pyplot_api.html

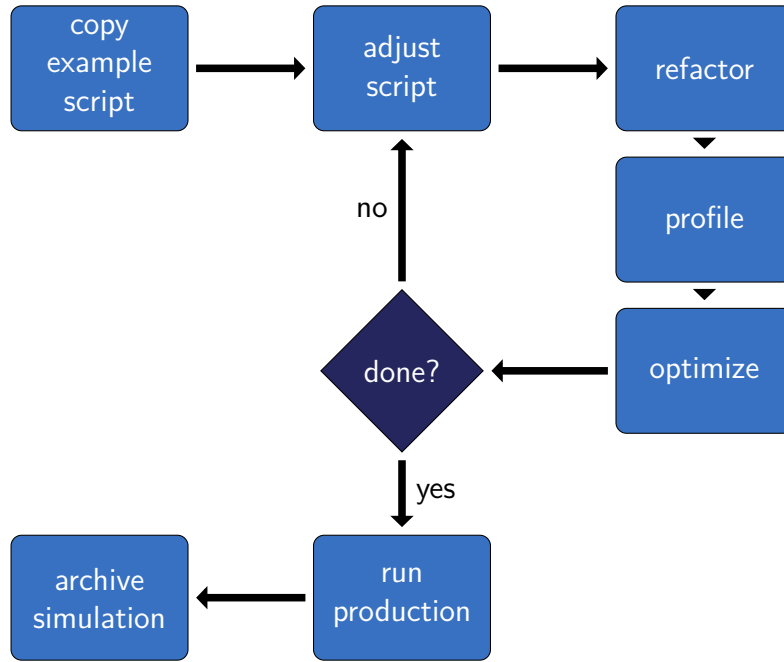


Figure 1.1: Workflow for designing a production script.

A solid background in math is nice, and surely helpful, but you can probably read and understand most of this book (up to part II) with only rudimentary mathematics and calculus. Similarly, a Ph.D. or other strong background in astrophysics or computer science will certainly make some of the material easier to digest, but we have successfully used AMUSE and this text with students at all levels, from freshman undergraduate to Masters to Ph.D. We even have users with a high-school background.

1.1.4 Alternative simulation environments

There are no real alternatives to AMUSE, at least not if you desire to perform multi-scale and multi-physics simulations. Here we mention a few projects with similar objectives that most closely comply to the listed requirements for a multi-purpose simulation environment. These include **Cactus**, a multiscale modeling framework used in **muscle3**, **FDPS**, and **MMSF**. We will briefly discuss each of these frameworks.

Cactus (Goodale *et al.*, 2003) is a general-purpose problem-solving environment for the physical sciences (Allen *et al.*, 2001). It is modular and supports parallel computing across the network with a variety of hardware. **Cactus** was last updated (version 4.12 of May 2022).² **Cactus** offers several advantages over other frameworks. It uses a component-based architecture, where the core functionality is provided by a central “*flesh*” and various “*thorns*” (modules) that can be independently developed, compiled, and loaded. The thorns can be parallelized, for multi-core architectures and distributed computing environments. Its high level of abstraction and interoperability with other tools simplifies the development of complex simulations. Using **Cactus** can be very rewarding, but understanding the details is cumbersome, and it may require

²see <http://www.cactuscode.org>

numerous recompilations to generate a working implementation.

FDPS stands for Framework for Developing Particle Simulators (Iwasawa *et al.*, 2016)³. It is built with much the same philosophy as AMUSE, but is much more coarse-grained. If FDPS were some sort of Lego (in software terms) for simulating self-gravitating systems, then AMUSE would be more like Duplo: big blocks that allow you to build an interesting colorful building with only a little effort.

FDPS is a high-performance, open-source library, written in C++ and designed to simplify and accelerate the development of large-scale, parallel particle-based simulation codes—such as N -body or SPH simulations—on distributed-memory supercomputers (Iwasawa *et al.*, 2016). *Petar* (Wang *et al.*, 2020), the N -body code used in Torch (Section 10.2.1), is written using FDPS. It provides a good example of how code written in FDPS can be embedded in AMUSE.

FDPS is optimized for building fast, scalable, single-physics particle simulation codes from scratch, with a focus on parallel performance, whereas AMUSE is designed for integrating and orchestrating multiple existing codes within a high-level, user-friendly Python environment, prioritizing flexibility and multi-physics capabilities. Researchers should choose between them based on whether their priority is raw computational performance for custom particle simulations (FDPS) or flexible, multi-physics coupling and ease of scripting (AMUSE). As with *Cactus*, FTSP also requires recompilation after changes are made.

Finally, we briefly mention the Multiscale Modeling and Simulation Framework (MMSF, Borgdorff *et al.*, 2013; Chopard *et al.*, 2014), which is based on a similar philosophy as that underlying AMUSE. MMSF systematically organizes and couples models at different scales, providing a rigorous and flexible approach to simulating complex, multi-scale systems. It is modular and is supported by formal languages and software tools for efficient implementation and execution. MMSF has been applied effectively in the *muscle3* environment (Veen & Hoekstra, 2020). However, in contrast to AMUSE’s structure of a driver script instructing a set of models to run and exchange information, it instead manages a set of distributed, mostly independent models which exchange messages via externally configurable connections, which block if a result they need is not yet available.

All three framework environments require recompilation before reuse, and are therefore not as suitable for experimentation as AMUSE. Each of them, however, is easier to tune to the appropriate hardware environment, but may lack native GPU support. Each package has its advantage and disadvantages. AMUSE stands out for its ease of use, breadth of included solvers, robust unit and data handling, and its focus on reproducible, modular, and flexible multi-physics simulations. AMUSE and FDPS are geared towards astrophysics, whereas MMSF, *Cactus*, and targets a broader range of applications.

1.1.5 Objective of this book

This book is about how to perform simulations of astrophysical phenomena. These simulations have the goal of mimicking nature, or at least certain aspects of it, in

³see: <https://github.com/FDPS/FDPS>.

order to better understand the universe. By the end of this book you will be able to perform many high-end simulations yourself. You will have learned how to set up initial conditions, how to design and develop a simulation environment to address your astrophysical question, how to perform the runs, and how to analyze the resulting data.

You can read this book in several ways. You can simply start at the beginning and work through to the end, or you may prefer to pick out the pieces you need in order to bootstrap your own research project. Either way, we recommend that you read the first couple of chapters, to understand how AMUSE works and some of its basic programming ideas.

1.1.6 What is missing from this book

This book is far from exhaustive. Many topics, such as data mining, high-performance and distributed computing, programming paradigms for graphics processing units, operating systems, and the effects of limited numerical precision are entirely separate disciplines that we can only direct you to. AMUSE contains far more functionality than we can describe. We will only briefly address the algorithms underlying many of the phenomena discussed. Our discussion of the underlying physics is also necessarily limited. This is not an astrophysics text—rather, our goal is to present just enough astrophysics to allow you to harness the modules we describe.

Of the many astrophysical phenomena we could include, we have decided to limit our scope to gravitational dynamics, stellar evolution, hydrodynamics, radiative transfer, and chemistry. We neglect a lot of physics, including asteroseismology, astrobiology, planetary geology, cosmology, YORP⁴, and tidal dissipative effects. These are not unimportant, but the book is finite, and new modules addressing such areas are constantly being added to the AMUSE framework.

1.1.7 Outline of the book

This book is divided into three parts. In Part I, Chapters 4 to 7, we discuss scripts designed to address single physical processes, for which only one numerical solver is required. A *minimal solver* is the simplest possible script needed to run a particular solver. In AMUSE, the low-level solvers—usually written in high-performance languages and often optimized for parallel architectures—are called *community codes*. When we include the AMUSE Python interface structure we generally use the more generic term *community module*. In practice, we often need additional modules for generating initial conditions, boundary values, or post-production analysis, but the main work in a minimal code is performed by a single community module. Minimal solvers run a particular code and stop after producing just enough information to gauge the quality of the solution. The solvers presented here should be viewed as examples to illustrate the AMUSE programming style and module interfaces, but they can also be used as a basis for further work.

⁴The Yarkovsky-O’Keefe-Radzievskii-Paddack effect (Rubincam, 2000) influences the spin of a minor body by reprocessing stellar radiation as thermal emission.

The next level of complexity is a *simple solver*, which is an extended version of a minimal solver. Simple solvers are meant to illustrate more complex and practical mono-physics applications. Within the AMUSE framework we can run a single code as a standalone program. Since AMUSE contains multiple solvers for each physical domain it addresses, this allows for detailed and instructive comparison between them. The ease with which initial conditions can be generated, and with which the user can change solvers and analyze the results makes running multiple solvers on the same problem a useful exercise. In this regard, the main advantage of using AMUSE is the unified way in which the input is parsed and delivered to, and results returned from, the solver. This enables the possibility of using exactly the same initial conditions for each of a set of numerical solvers, and analyzing the resulting data in a uniform way.

In Part III, Chapters 8 and 9, we discuss elementary and hierarchical code coupling strategies, followed by Chapter 10, in which we present a few example cases for multi-scale and multi-physics simulations.

In the last part of the book, Part III, we dwell on expert knowledge and discuss the subtleties of multi-scale and multi-physics simulations. In the last chapter (Chapter 10), we discuss possible expansions and other packages derived from AMUSE.

The chapters are followed by several appendices. In Appendix A we further discuss the fundamental concepts of AMUSE, and give details about its internal operation. Important, but often not considered multi-scale or multi-physics are the generation of initial conditions, which we discuss in Chapter 3. How to add your own code to AMUSE is discussed in Appendix B. The last appendix includes a brief Python primer (Appendix C).

We have adopted this presentation order for its steady increase in physical complexity and its systematic introduction of more and more sophisticated AMUSE capabilities.

1.2 A brief history of simulations in astrophysics

The first recordings of theoretical astronomy date back to the Egyptian 18th dynastic (1550–1292 BCE) calculations by pharaoh Hatshepsut’s architect Senmut; portions of the murals found in his tomb at Deir el-Bahri can be found at <http://www.metmuseum.org> (Dorman, 2006). Figure 1.2 presents part of this mural. Such calculations enabled astronomers to recognize structure and describe patterns in the heavens, an aspect of astronomy that is still relevant today.

Although the Egyptian civilization did not show much academic interest in the heavens, this changed dramatically when the Babylonian civilization emerged. Around 350 to 50 BCE, they already showed a keen interest in mathematics and algorithms. This is demonstrated by in several clay tablets with cuneiform writing, such as the one presented in Figure 1.3, which shows a calculation of Jupiter’s orbital integration on the sky. The complexity of this calculation, due to the epicycles the planet makes when its orbit is folded in with the Earth’s and subsequently projected on the sky, requires a second order integration. The method depicted in Figure 1.3 resembles a Leap-frog or Verlet integrator. Although we do not know who wrote the tablet, or who designed the algorithm, it is clear that the Babylonians were already aware of 2nd

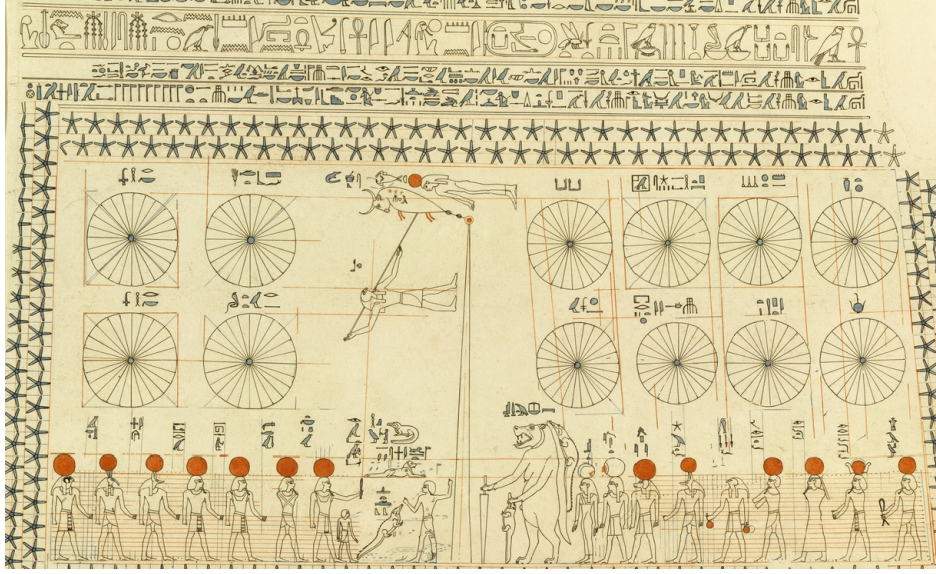


Figure 1.2: Reproduction of the lower part of the 2nd tomb of Senenmut’s stela, dated around 1479-1458 BCE (accession number MMT:48.105.52 (TT353), Figure 27 of [Wilkinson, 1979](#), GNU General Public License, Rogers Fund, 1948). This unique mural shows, apart from all the Egyptian gods, the most important constellations and a much debated conjunction between the Sun and the Pleiades star cluster. The original photograph was presented in [Dorman \(2006, 1991\)](#).

order error-reducing algorithms ([Ossendrijver, 2016](#)). It would be better to call this the Babylonian integrator.

Many centuries later, modern astronomy started with the discovery of the first refracting telescope, built by Hans Lippershey in 1608, which had a collecting area of about 1 cm^2 . Galileo Galilei reported the first astronomical applications in 1609, using a telescope of similar design. In the 400 years since that momentous first application of new technology, the collecting areas of the largest research telescopes have increased by a factor of about 10^6 , becoming the 10-m class telescopes in widespread use today.

Digital computers were introduced in the late 1930s ([Zuse, 1967, 1993](#)), and the first commercial programmable model, ENIAC, achieved in 1945 a computational speed equivalent to about 200 modern floating point (double precision) operations per second (flops). (We say “equivalent” because ENIAC was essentially a single precision machine.) Today the fastest supercomputers achieve speeds exceeding 10^{18} (Exa) flops, a speedup of a factor of roughly 10^{15} in raw performance in just eight decades. Thus, while observing technology has grown impressively but relatively slowly over the past four centuries, computer scientists have witnessed a technological explosion in living memory.

The ongoing revolution in the availability of digital computers has led to an entirely new branch of research in which key facilities are not confined to high mountain tops in Hawaii or the Chilean Andes, but more often in the room next door. Astronomers quickly realized that they could use computers to archive, process, analyze, and mine the copious amounts of data taken by observing campaigns. However, arguably the biggest impact on the way astronomers pursue their scientific questions has been in numerical simulations of astrophysical phenomena.



Figure 1.3: Photo of text A (lines 1 to 7) of the Babylonian clay tablet from 350 to 50 BCE with the mathematical tables to predict Jupiter’s orbit on the sky. The original photograph was presented as figure 1 in [Ossendrijver \(2016\)](#). The tables is about 4×4 cm.

Astronomical research lends itself naturally to computation because it is characterized by large ranges in temporal and spatial scales, complex and coupled nonlinear processes, and the extreme conditions found in interstellar space. With the appearance of digital computers it became possible to study processes in the intergalactic vacuum, hot plasmas at stellar surfaces, billion-year time scale processes, and black-hole physics, none of which can be studied in Earth-based laboratories. The major current limitation to studying the universe by means of computation lies in software, and in particular our limited understanding of algorithms that can solve multiscale, multiphysics problems. AMUSE is an attempt to address these issues.

Much help in generating AMUSE scripts can be expected from artificial intelligence via Large Language Models (LLM). One can simply ask a LLM to write you a script using AMUSE, and since these models are trained on the book’s first version examples, it generally does a reasonable job. They tend to be a bit confused about channels and bridges, though, which might indicate that our explanations are not always as easily apprehended by an AI.

1.2.1 The first simulation experiments

One of the first astrophysical simulations was carried out in 1941 by Erik Holmberg, a graduate student at Lund University who had access to a number of photocells.

Using the fact that the intensity of radiation drops off as the inverse square of the distance, like the force of gravity, he devised an experiment to simulate two colliding galaxies. The initial setup for a single “galaxy” in Holmberg’s experiment is presented in Figure 1.4. The progress of his experiment is shown in Figure 1.5. The arrows indicate the directions in which the galaxies move with respect to one another, and how they rotate. This experiment made Holmberg one of the founders of what is now an immense body of astronomical research in gravitational dynamics, some of it reviewed by [Binney & Tremaine \(1987\)](#).

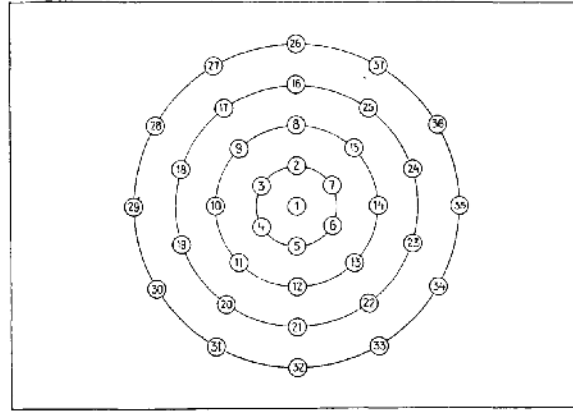


Figure 1.4: Copy of Figure 3 from [Holmberg \(1941\)](#), presenting the initial conditions for a single galaxy in Holmberg’s simulation of two colliding galaxies. (Copyright AAS. Reproduced with permission.)

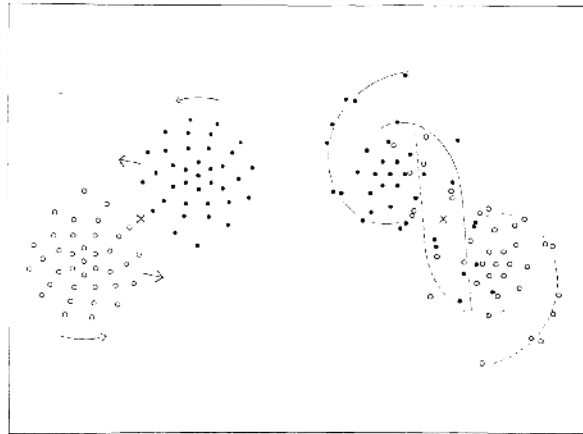


Figure 1.5: Copy of Figure 4a from [Holmberg \(1941\)](#), presenting two snapshots of the results from his calculations. Both snapshots were taken at the moment the two galaxies were separated by twice their radii, one before the interaction (left) and one after the first close approach (right). (Copyright AAS. Reproduced with permission.)

Holmberg had no access to a computer, so all of his calculations were performed by hand. His article ([Holmberg, 1941](#)) is interesting in many respects, but unclear on exactly how the forces (photon counts in his case) were used to update the positions of the particles (the light bulbs). Today, in a comparable paper, we might state that the integration was performed using (say) a 4th-order Hermite predictor-corrector scheme

with direct force evaluation and zero softening (see Makino & Aarseth, 1992). The choice of algorithm (in this case the Hermite scheme) is an important part of the solution. Although this level of detail might sound excessive, you will better appreciate such considerations by the time you have read Chapter 4.

1.3 Software used in this book

1.3.1 Motivation for a homogeneous software environment

Developing high-quality simulation software is relatively straightforward when the physical scope of the problem is limited. The task becomes considerably more difficult once the application spans multiple physical domains, or if the dynamic range in temporal or spatial scales exceeds the precision of the hardware. Successful examples of complex multiphysics software in computational astrophysics are NBODY7 (Aarseth, 1999, 2011), FLASH (Fryxell *et al.*, 2000, 2010), and GADGET (Springel *et al.*, 2001; Springel, 2000), which have broad user bases and can be applied to a wide variety of problems. (Note that, where possible, we provide two references for each software implementation, one to the publication in which the package was first described, and one to the Astrophysics Source Code Library at <http://ascl.net/> or ZENODO <https://zenodo.org/>) Some of these codes address relatively narrow ranges of physics and are limited by the specific numerical algorithms or data topology adopted to solve the underlying equations, but others, such as FLASH, are highly modular in their internal organization, in some ways similar to AMUSE. Most of these software is hosted in US servers. In the near future we expect to move all our activities to a less autocratic location.

Figure 1.6 presents typical temporal and spatial scales for a number of astronomical phenomena. The ellipses indicate specific areas of interest. The colors indicate physical domains: gravity (blue), stellar evolution (red), hydrodynamics (yellow), and radiative processes (green). Several domains are identified by name, such as the collapse of a giant molecular cloud (top right in yellow), and the range of habitable planet orbits (blue circle at lower left). Gravity and hydrodynamics span a broad range of temporal and spatial scales, whereas radiative processes and stellar evolution exhibit much narrower ranges. Note that both spatial and temporal scales span a range of about 14 orders of magnitude, from galactic orbits at one extreme to stellar mergers or intricate orbital dynamics at the other.

Many existing simulation codes provide excellent solvers within their specific physical domains, but astrophysical problems often require solvers for a wide variety of domains. For example, understanding the ejection of primordial gas from an embedded star cluster, born during a galaxy merger, requires simultaneous treatments of collisional stellar dynamics (the protostellar cluster), collisionless stellar dynamics (the galaxy merger), hydrodynamics (interstellar gas), stellar evolution (embedded proto-stars and stars), and radiative processes (feedback between stars and the interstellar gas). All these processes occur on comparable temporal and spatial scales.

Instead of designing new algorithms to solve all of these problems simultaneously, we adopt the philosophy that each of the physical domains can be solved separately, then

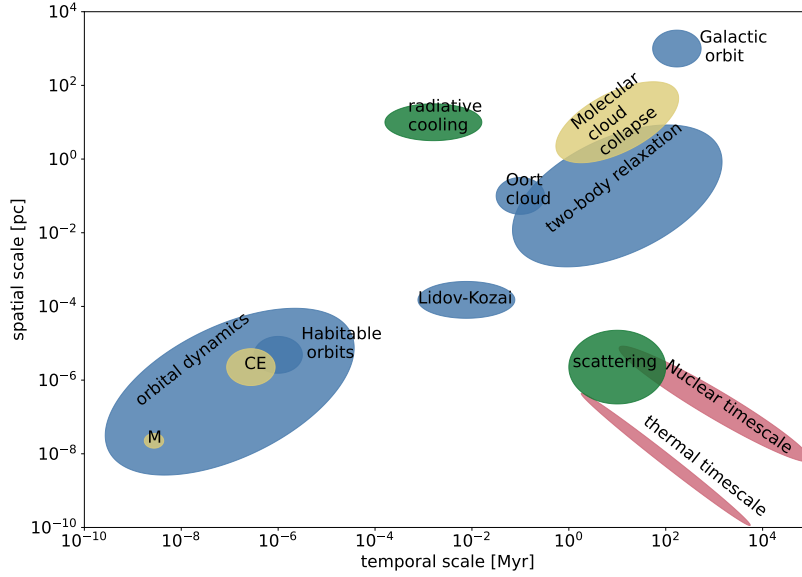


Figure 1.6: A number of typical scales, for gravitational dynamics (blue), stellar evolution (red), hydrodynamics (yellow), and radiative transfer (green), are indicated at several locations in the plane of temporal and spatial scales. Here several small scales (mergers indicated by *M*) and common envelope evolution (indicated by *CE*) are embedded within the orbital dynamics scale ellipse.

combined into a consistent solution at discrete intervals in time and space. Validation of such a simulation could subsequently be accomplished by recomputing the model with a different solver for each of the physical domains. In this way we can study the convergence (or the absence thereof) of the solution with a sequence of numerical implementations. The Astrophysical Multipurpose Software Environment (AMUSE; Portegies Zwart, 2011; Portegies Zwart *et al.*, 2013) allows a user to combine existing solvers to build new applications that can be combined again to study increasingly complex problems. This enables the growth of multiphysics and multiscale application software in a hierarchical fashion, and allows testing of each intermediate step as the complexity of the software increases.

AMUSE has a hierarchical component architecture that encapsulates dynamically shared libraries for simulating stellar evolution, gravitational dynamics, (magneto)-hydrodynamics, radiative processes, and also has limited support for astrochemistry. Applications are able to use some or all of these resources simultaneously, and can exchange data in a unique and deterministic way. With this hybrid simulation environment it becomes possible to study multiphysics processes operating on a broad range of length and time scales.

1.3.2 Choice of programming languages

We have adopted Python (van Rossum, 1995a,b,c,d) as the implementation language for the AMUSE framework and high-level management functions, including bindings

to the communication interface. The earlier version of this book used Python2.7, but we have since migrated to Python3 [Van Rossum & Drake \(2009\)](#) (as of writing 3.8 or higher)⁵. The choice of Python is motivated by its flexibility, broad acceptance in the scientific community, object oriented design, and ability to allow rapid prototyping, which shortens the software development cycle and enables easy access to the community code in the community module, albeit at the cost of reduced performance ([Portegies Zwart, 2020](#)). However, the entire AMUSE framework is organized in such a way that relatively little computer time is actually spent in the framework itself—most time is (or should be) spent in the community code(s). The overhead from Python compared to a compiled high-level language is typically $\lesssim 10\%$, and often much less ([Pelupessy et al., 2013](#)).

As discussed further in Paragraph [B.1.1.2](#), our implementation of the communication layer in AMUSE uses the standard Message Passing Interface protocol (MPI; [Gropp et al., 1996](#)) and SmartSockets ([Maassen & Bal, 2007](#)). Normally MPI and SmartSockets are used for communication between compute nodes on parallel distributed-memory systems or within computational grids. However, in AMUSE they are used as communication channels between *all* processes, whether or not they reside on the same node as the control script, and whether or not they are themselves parallel.

AMUSE community codes are written in a variety of languages, reflecting the programming backgrounds and preferences of their authors. As a result, for each community code there are two interface functions. The one on the AMUSE side is written in Python and is called the proxy. The interface on the community code side is called the partner and is written in the native language of that code. These two interfaces communicate via one of the message-passing protocols just described. Thus our only real restriction on supported languages is the requirement that the community code is written in a language with MPI or Smart Sockets bindings, such as C ([Ritchie, 1993b,a](#)), C++, ([Stroustrup, 2013](#)), Java ([Gosling et al., 1996, 2014](#)), and members of the FORTRAN family ([Backus, 1978](#)).

We emphasize that, from the user perspective, the details of the interface and communication protocols are invisible. These details are central to the AMUSE design, but most high-level users are blissfully unaware of them. To the user, a Python AMUSE script looks just like a traditional serial program.

1.3.3 Design of AMUSE

The AMUSE project goal stipulated three basic functionalities:

- (1) a homogeneous, physically motivated interface for existing astronomical simulation codes;
- (2) the incorporation of multiple community codes from four fundamental domains (stellar evolution, gravitational dynamics, hydrodynamics, and radiative transfer); and

⁵Python version 3.13 was the most recent release while writing this.

- (3) the ability to design new simulation experiments by combining one or more of the community codes in various ways.

In this way researchers can concentrate on physically relevant details, such as the calling sequence for the various physical domains, without having to worry about the many technical details.

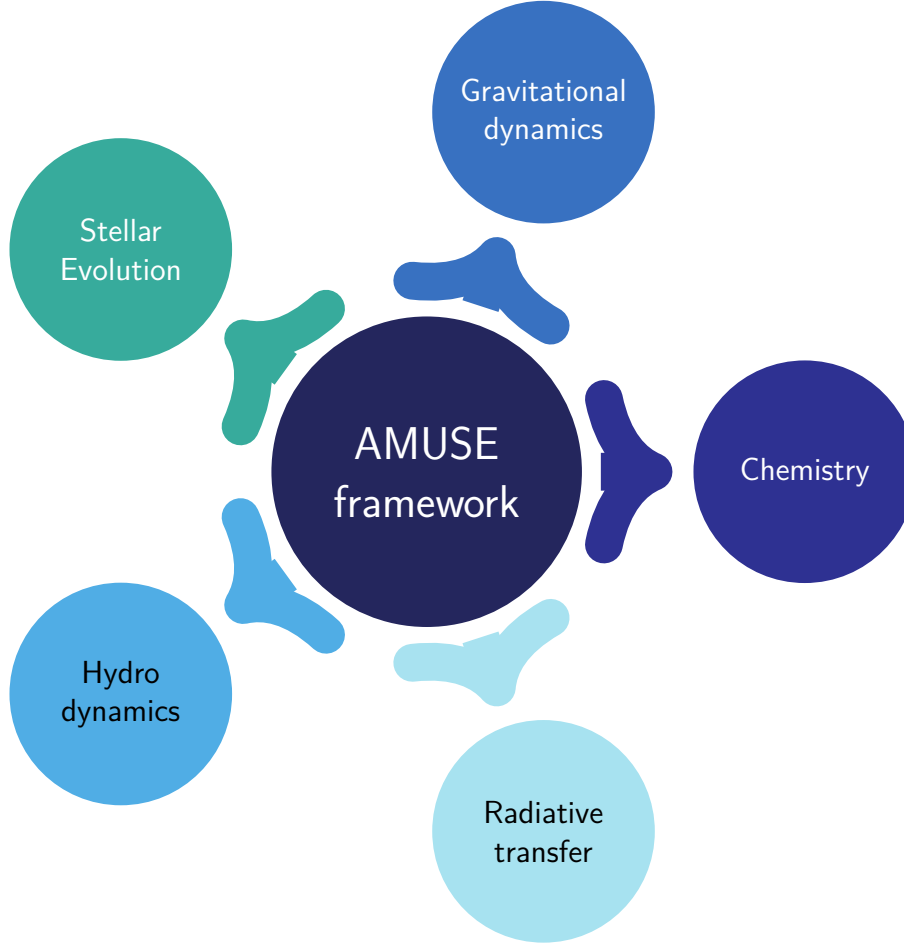


Figure 1.7: Multiscale and multiphysics codes are connected disciplines, as depicted here in the domain–application plane.

Each of the existing community codes in AMUSE is embedded in an interface layer that realizes communication between the AMUSE framework and a community code. One important advantage of this strict separation is that it explicitly prohibits short-cut communication between independent subroutines, greatly diminishing the opportunities to create “spaghetti code”⁶. In addition, it allows us to manipulate data as it moves between the community code and the framework. One such data manipulation is the conversion of units, as discussed in Section 3.4.1. The handling of community-code dependent parameters are largely managed by the framework, as is the acquisition and mining of simulation data. Figure 1.7 illustrates schematically

⁶In our opinions the term “ramen code” would be more appropriate.

the philosophy of AMUSE, with the central AMUSE framework surrounded by its four fundamental physical solvers.

New hybrid codes can be written in AMUSE by combining community codes using Python. An AMUSE Python script for solving a particular problem prescribes the order in which events are handled and specifies the numerical solver that should be used to resolve each event. Once initial conditions are generated and the simulation is completed, the resulting data can be stored or further analyzed using the tools available in AMUSE, or developed by the user. The technical problems associated with coupling codes, setting boundary conditions, specifying run-time parameters, and guaranteeing that the various elements communicate in the proper units are handled entirely by AMUSE. A thorough understanding of the limitations of each of the incorporated codes is important for assessing the quality of the simulation, but the burden of managing the technical details is not forced on the user.

We have demonstrated modularity and interchangeability of the AMUSE community codes for solving the basic physical processes described above. The user can select from a variety of gravitational N -body solvers (symplectic N -body codes, direct N -body codes, and tree codes) by changing a single word in the script. The same holds for stellar evolution codes (parameterized and Henyey codes), hydrodynamics solvers (smooth particle hydrodynamics and adaptive mesh refinement codes), and radiative transfer codes (Monte Carlo photon transport codes, and tessellated photon flow solvers). The modularity, and how a complex cascade of numerical solvers can grow to an end product, is illustrated in Chapter 10, and in Part III. This cascade of jargon may sound like Jabberwocky (Carol, 1871) right now, but we hope that much of it will eventually make sense by the time you reach the end of this book.

1.3.4 Terminology

The basic structure of the AMUSE framework is illustrated in Figure 1.8. The figure is subdivided into three blocks. The top block is mostly user written, the middle block implements the interfaces onto the community codes, and the bottom block is the community modules and the actual community source code, which is connected to the rest of the framework via MPI or sockets.

The components can be described as follows:

- The Python *user script* (top layer in Figure 1.8), which addresses a specific physical problem or set of problems, in the form of system-provided or user-written Python commands that serve as the user interface onto the framework. This is the only part of a production script that needs to be written by the user. The coupling between community codes is implemented in this layer, with the help of support classes for converting units and data structures provided by AMUSE (see Section 2.3.1).
- The *community module*, comprising three key elements:
 - The *manager* (second layer in top block), which provides an object-oriented interface onto the communication layer via a suite of system-provided utility

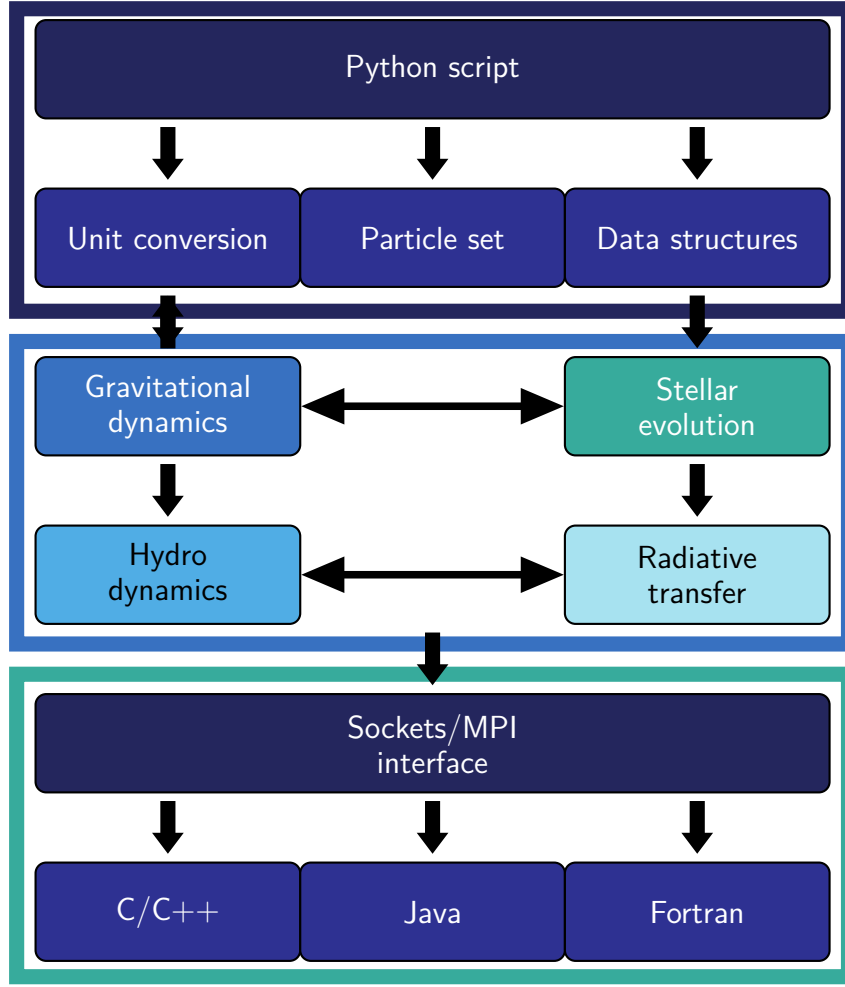


Figure 1.8: General structure of the AMUSE framework. The user writes the Python script (top layer). The lower layers in the first block are responsible for units and data structures. The framework in the second block represents the interfaces to the community modules (dark blue for gravitational dynamics, green for stellar evolution, funny blue for hydrodynamics and radiative transfer in light blue to conform with the color coding of Figure 1.7), which connect via sockets or MPI (third block) to the community codes at bottom. Information flows downward to the community modules, as indicated by the arrows, as well as upward, back to the Python script.

functions. This layer handles unit conversion, and also contains the state engine and the associated data repository, all of which are required to guarantee consistency of data across modules. This layer is generic—it is not specific to any particular problem or physical domain.

- The *communication layer* (second block), which realizes the bidirectional communication between the manager and the community code layer. This is implemented via a proxy and an associated partner, which together provide the connection to each community code.
- The *community code layer* (third block), which contains the actual community codes and implements control and data management operations on them. Each piece of code in this layer is domain-specific, although the code may be designed to be very general within its particular physical domain.

1.4 Installing AMUSE

Before you continue reading, we advise you to install the AMUSE package. Here is a brief overview of the installation steps. We assume that you are working in a Linux or macOS environment. We don't recommend or support AMUSE under Windows, except within a Linux environment such as Windows subsystem for Linux⁷ or Multipass⁸. For the following we assume that you have a basic understanding of your operating system and the command line.

To install AMUSE, we strongly recommend creating a new virtual environment, either via `conda`⁹ or via Python3's `venv`¹⁰. AMUSE can be installed either as a package or from source in either. Other virtual environments could work equally well, but are not supported by us at the time of writing.

1.4.1 Installing AMUSE in a virtual environment

Here we will start from a `python venv` or `conda` environment on a Ubuntu linux laptop. This tutorial was written on a fresh Ubuntu 24.04.2 LTS (6.5.0-1027-oem) installation on a Lenovo P14s laptop. This is not a particularly high-end computer, but it should suffice for all example calculations in this book. If only, because all the example calculations in this book are generated with this same machine. We would like to emphasize here that the revenues we receive for writing this book is by far insufficient to purchase such a laptop.

Open a terminal and copy the lines below to create an environment for your project (note that the leading characters only indicates the terminal prompt, and it may be different on your computer. The instructions are written for `bash` or another Bourne shell.

```
%> git clone git@github.com:amusecode/amuse.git .  
%> cd amuse  
%> ./setup
```

This downloads the AMUSE source code and runs the AMUSE setup script. This script will detect the environment you are using and guide you through the installation process.

Since the details of the installation process may evolve over time, we do not print the full instructions here.

However, if you feel uncertain about what to do, try:

```
%> ./setup help
```

which will give you some options.

After following the instructions, the AMUSE framework should be built, and probably most of the packages and libraries.

⁷https://en.wikipedia.org/wiki/Windows_Subsystem_for_Linux

⁸<https://multipass.run>

⁹<https://conda-forge.org/download>

¹⁰<https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments>

In case something is broken, you can visit our GitHub issue tracker (<https://github.com/amusecode/amuse/issues>) and ask for help.

1.4.1.1 Testing your AMUSE installation

After a successful installation, we encourage you to run a standard set of tests. The general tests can be run quite similarly to installing using the setup routine

```
%> ./setup test amuse-framework
```

for testing the framework, or

```
%> ./setup test ph4
```

testing an individual code, such as ph4 in this case.

1.4.2 Running the examples

Examples in this book, and throughout the entire AMUSE framework can be run by invoking the python `examples` package

```
%> ./python -m amuse.examples.{example_name}
```

Here `example_name` is the name of the example.

To make this work, one first has to install the amuse example package through pip.

```
%> pip install amuse-examples
```

And then run

```
%> ./python -m amuse.examples.sun_venus_earth
```

will run the example script `sun_venus_earth` and reproduce Figure 3.4.

1.4.3 Running AMUSE

By now, you should have built and tested AMUSE, as described in Section 1.4, and are probably wondering “What can AMUSE do for me?” This section will get you started. If you’re new to Python you’ll find the official Python documentation¹¹ a valuable resource. Most of the scripts in this book were rewritten with at least Python version 3.7 in mind, but much of the coding will remain up-to-date with the current version 3.12 and higher.

You can run AMUSE interactively from Python or iPython:

```
%> python
```

Once Python has started, you can load AMUSE by

```
from amuse.lab import *
```

¹¹<https://docs.python.org/3>

This line is a lazy but very quick way to import most of the AMUSE package in a single call. However, we generally do not advise it because it pollutes the Python name space. Rather, we recommend that modules are imported from AMUSE individually, and in the examples that follow this is how we will present source code.

The code examples and snippets presented in the book are designed to be cut and pasted into your Python interpreter or your file editor. However, due to limitations of online book formatting, whitespace may not be conserved when doing so. To facilitate using the code examples directly, we provide an `amuse-examples` package that contains the source code for all examples in this book as well as all the generated figures. Once installed the examples can be imported in the following way (using the `ae` namespace as shorthand for `amuse.examples`):

```
import amuse.examples as ae
```

The complete source code of an example (e.g. `gravity_minimal`) can be seen with the `ae.print_example` function:

```
ae.show("gravity_minimal")
```

If you are using AMUSE from a Jupyter notebook, you can copy the example directly to a new cell, ready for editing:

```
ae.to_cell("gravity_minimal")
```

To run an example with default settings, you can use the `run` function:

```
ae.run("gravity_minimal")
```

We hope that these examples and code snippets encourage you to experiment and initiate interesting research projects.

1.4.3.1 Running AMUSE without network access

Running AMUSE without a network connection is curiously not always supported by your favorite operating system. The underlying reason hides in the requirement for MPI to have a network connection. This can be solved by using the smart sockets connection channel, or by using a loopback network. The latter can be done with the following command line:

```
%> mpirun -n 1 --mca btl_tcp_if_include lo python -m amuse.examples.gravity_minimal
```

It's a rather complex command, but fundamentally, it boils down to running the MPI command `mpirun` using a single core with the specific command for process binding for the tcp protocol on your local network. This so-called `MCA` command (`-mca`) can be invoked in `mpirun` with the arguments `btl_tcp_if_include` and an indicator for the local network (`lo`). This is a whole list of jargon, which may be hard to parse if you barely installed the AMUSE package, but maybe it shows that some of the things we do are not always as well supported by local operating systems as we would wish. There are quite a few other complex message-passing and scheduling issues if you desire to

run AMUSE on larger machines, but since the architectures of such machines change within the mean time scale of a PhD (and on the time scale of a new release of this book), we will not dwell further onto this topic.

Bibliography

- Aarseth, Sverre J. 1999. From NBODY1 to NBODY6: The Growth of an Industry. *PASP*, **111**(765), 1333–1346.
- Aarseth, Sverre J. 2011 (Feb.). *NBODY Codes: Numerical Simulations of Many-body (N-body) Gravitational Interactions*. Astrophysics Source Code Library, record ascl:1102.006.
- Allen, Gabrielle, Bengert, Werner, Dramlitsch, Thomas, Goodale, Tom, Hege, Hans-Christian, Lanfermann, Gerd, Merzky, André, Radke, Thomas, & Seidel, Edward. 2001. Cactus grid computing: Review of current development. *Pages 817–824 of: Euro-Par 2001 Parallel Processing: 7th International Euro-Par Conference Manchester, UK, August 28–31, 2001 Proceedings* 7. Springer.
- Backus, John. 1978. *The History of Fortran I, II, and III*. New York, NY, USA: Association for Computing Machinery. Page 2574.
- Beck, K., & Andres, C. 2004. *Extreme Programming Explained: Embrace Change (2Nd Edition)*. Addison-Wesley Professional.
- Binney, James, & Tremaine, Scott. 1987. *Galactic dynamics*.
- Borgdorff, Joris, Falcone, Jean-Luc, Lorenz, Eric, Bona-Casas, Carles, Chopard, Bastien, & Hoekstra, Alfons G. 2013. Foundations of distributed multiscale computing: Formalization, specification, and analysis. *Journal of Parallel and Distributed Computing*, **73**(4), 465–483.
- Calaprice, A. 2002. *Dear Professor Einstein: Albert Einstein's Letters to and from Children*. Prometheus.
- Carol, L. 1871. *Through the Looking-Glass, and What Alice Found There*. Macmillan.
- Chopard, Bastien, Borgdorff, Joris, & Hoekstra, Alfons. 2014. A framework for multi-scale modelling. *Philosophical transactions. Series A, Mathematical, physical, and engineering sciences*, **372**(08).
- Dorman, .F. 2006. The career of Senenmut from Hatsehsut. *Pages 107–109 of: From Queen to Pharaoh Ed. C. Roehrig*.
- Dorman, P. F. 1991. *The Architecture and Decoration of Tombs 71 and 353*. Metropolitan Museum of Art Egyptian Expedition, New York.
- Fryxell, B., Olson, K., Ricker, P., Timmes, F. X., Zingale, M., Lamb, D. Q., MacNeice, P., Rosner, R., Truran, J. W., & Tufo, H. 2000. FLASH: An Adaptive Mesh Hydrodynamics Code for Modeling Astrophysical Thermonuclear Flashes. *ApJS*, **131**(1), 273–334.
- Fryxell, B., Olson, K., Ricker, P., Timmes, F. X., Zingale, M., Lamb, D. Q., MacNeice, P., Rosner, R., Truran, J. W., & Tufo, H. 2010 (Oct.). *FLASH: Adaptive Mesh Hydrodynamics Code for Modeling Astrophysical Thermonuclear Flashes*. Astrophysics Source Code Library, record ascl:1010.082.
- Goodale, Tom, Allen, Gabrielle, Lanfermann, Gerd, Massó, Joan, Radke, Thomas, Sei-

- del, Edward, & Shalf, John. 2003. The Cactus Framework and Toolkit: Design and Applications. *Pages 197–227 of: Palma, José M. L. M., Sousa, A. Augusto, Don-garra, Jack, & Hernández, Vicente (eds), High Performance Computing for Computational Science — VECPAR 2002*. Berlin, Heidelberg: Springer Berlin Heidelberg.
- Gosling, J., Joy, B., & Steele, G.L. 1996. *The Java Language Specification*. 1st edn. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc.
- Gosling, J., Joy, B., Steele, G. L., Bracha, G., & Buckley, A. 2014. *The Java Language Specification, Java SE 8 Edition*. 1st edn. Addison-Wesley Professional.
- Gropp, William, Lusk, Ewing, Doss, Nathan, & Skjellum, Anthony. 1996. A high-performance, portable implementation of the MPI message passing interface standard. *Parallel Computing*, **22**(6), 789–828.
- Holmberg, Erik. 1941. On the Clustering Tendencies among the Nebulae. II. a Study of Encounters Between Laboratory Models of Stellar Systems by a New Integration Procedure. *ApJ*, **94**(Nov.), 385.
- Hunter, John D. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science and Engineering*, **9**(3), 90–95.
- Iwasawa, M., Fukushige, T., Nitadori, D., & Makino, J. 2016. Framework for Developing Particle Simulators (FDPS): Parallel Particle-Mesh Simulations Made Easy. *Publications of the Astronomical Society of Japan*, **68**(3), 54.
- Maassen, J., & Bal, H. E. 2007. Smartsockets: solving the connectivity problems in grid computing. *Pages 1–10 of: HPDC '07: Proceedings of the 16th international symposium on High performance distributed computing*. ACM. 1272368.
- Makino, Junichiro, & Aarseth, Sverre J. 1992. On a Hermite Integrator with Ahmad-Cohen Scheme for Gravitational Many-Body Problems. *Publ. Astr. Soc. Japan*, **44**(Apr.), 141–151.
- Ossendrijver, M. 2016. Ancient Babylonian astronomers calculated Jupiter’s position from the area under a time-velocity graph. *Science*, **351**(Jan.), 482–484.
- Pelupessy, F. I., van Elteren, A., de Vries, N., McMillan, S. L. W., Drost, N., & Portegies Zwart, S. F. 2013. The Astrophysical Multipurpose Software Environment. *A&A*, **557**(Sept.), A84.
- Portegies Zwart, S., McMillan, S. L. W., van Elteren, E., Pelupessy, I., & de Vries, N. 2013. Multi-physics simulations using a hierarchical interchangeable software interface. *Computer Physics Communications*, **183**(Mar.), 456–468.
- Portegies Zwart, Simon. 2011 (July). *AMUSE: Astrophysical Multipurpose Software Environment*. Astrophysics Source Code Library, record ascl:1107.007.
- Portegies Zwart, Simon. 2020. The ecological impact of high-performance computing in astrophysics. *Nature Astronomy*, **4**(Sept.), 819–822.
- Portegies Zwart, Simon, McMillan, Steve, Harfst, Stefan, Groen, Derek, Fujii, Michiko, Nualláin, Breannán Ó., Glebbeek, Evert, Heggie, Douglas, Lombardi, James, Hut, Piet, Angelou, Vangelis, Banerjee, Sambaran, Belkus, Houria, Fragos, Tassos, Fregeau, John, Gaburov, Evghenii, Izzard, Rob, Jurić, Mario, Justham, Stephen, Sottoriva, Andrea, Teuben, Peter, van Bever, Joris, Yaron, Ofer, & Zemp, Marcel. 2009. A multiphysics and multiscale software environment for modeling astrophysical systems. *New Astron.*, **14**(4), 369–378.
- Ritchie, D. M. 1993a. The Development of the C Language. *SIGPLAN Not.*, **28**(3),

- 201–208.
- Ritchie, D.M. 1993b. The Development of the C Language. *Pages 201–208 of: The Second ACM SIGPLAN Conference on History of Programming Languages*. HOPL-II. New York, NY, USA: ACM.
- Rubincam, David Parry. 2000. Radiative Spin-up and Spin-down of Small Asteroids. *Icarus* , **148**(1), 2–11.
- Springel, Volker. 2000 (Mar.). *GADGET-2: A Code for Cosmological Simulations of Structure Formation*. Astrophysics Source Code Library, record ascl:0003.001.
- Springel, Volker, Yoshida, Naoki, & White, Simon D. M. 2001. GADGET: a code for collisionless and gasdynamical cosmological simulations. *New Astron.* , **6**(2), 79–117.
- Stroustrup, B. 2013. *The C++ Programming Language*. 4th edn. Addison-Wesley Professional.
- van Rossum, Guido. 1995a (Apr.). *Extending and embedding the Python interpreter*. Report CS-R9527. pub-CWI, pub-CWI:adr.
- van Rossum, Guido. 1995b (Apr.). *Python library reference*. Report CS-R9524. pub-CWI, pub-CWI:adr.
- van Rossum, Guido. 1995c (Apr.). *Python reference manual*. Report CS-R9525.
- van Rossum, Guido. 1995d (Apr.). *Python tutorial*. Report CS-R9526.
- Van Rossum, Guido, & Drake, Fred L. 2009. *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace.
- Veen, Lourens E., & Hoekstra, Alfons G. 2020. Easing Multiscale Model Design and Coupling with MUSCLE 3. *Pages 425–438 of: Krzhizhanovskaya, Valeria V., Závodszky, Gábor, Lees, Michael H., Dongarra, Jack J., Sloot, Peter M. A., Brissos, Sérgio, & Teixeira, João (eds), Computational Science – ICCS 2020*. Cham: Springer International Publishing.
- Wang, Long, Iwasawa, Masaki, Nitadori, Keigo, & Makino, Junichiro. 2020. PETAR: a high-performance N-body code for modelling massive collisional stellar systems. *MNRAS* , **497**(1), 536–555.
- Wilkinson, C.K. 1979. Egyptian Wall Paintings. *Pages 24–25 of: The Metropolitan Museum’s Collection of Facsimiles*. The Metropolitan Museum of Art Bulletin, vol. 36.
- Zuse, K. 1967. Rechnender Raum. *Elektronische Datenverarbeitung*, **8**, 336–344.
- Zuse, K. 1993. *The Computer – My Life*.